HBALS: A Hybrid Bees Algorithm with 2-opt Local Search for the Travelling Salesman Problem, Evaluated by Ablation and Equal-Budget Comparison

Erfan Khadiv*

Department of Computer and Information Technology, Islamic Azad University, Ardabil Branch, Ardabil, Iran

*Corresponding author: erfan.khadiv.dev@gmail.com

Preprint — not peer reviewed.

Abstract

The travelling salesman problem, finding the shortest closed route through a set of cities, is a standard test for optimisation algorithms, and nature-inspired methods are often combined with local search to solve it. We study HBALS, a bees-algorithm hybrid that starts from nearest-neighbour tours and applies 2-opt local search with different intensity at elite and ordinary sites. To make the comparison fair, every method uses the same local search and the same budget of move evaluations. On 12 benchmark instances with 51–198 cities, HBALS had the best average rank and outperformed memetic genetic and ant algorithms, multi-start 2-opt and a plain bees algorithm (Wilcoxon signed-rank test, Holm-adjusted $p = 0.0024$), but did not differ from iterated local search ($p = 0.13$); on two of three larger instances, iterated local search was better. An ablation study showed that local search at elite sites and a single-swap perturbation drive performance, whereas an adaptive neighbourhood rule was harmful and three further components had no measurable effect. Stopping runs after 20 non-improving iterations roughly tripled the average gap. Code and data are openly available.

Keywords: travelling salesman problem; bees algorithm; memetic algorithm; local search; ablation study; metaheuristics

Introduction

The travelling salesman problem (TSP) asks for the shortest closed tour that visits every city of a given set exactly once. It is a classical combinatorial optimisation problem[1] and a standard benchmark for new algorithms, with applications in logistics, scheduling, manufacturing and genome sequencing[2]. Exact methods become impractical for large instances because the problem is NP-hard, so heuristics and metaheuristics are widely used[1,3].

The bees algorithm (BA) is a population-based metaheuristic inspired by the foraging of honeybees[4,5]. It allocates more search effort to the best (elite) and to other promising (selected) sites, while scout bees keep exploring[6,7]. Recent work has proposed reduced-parameter and single-parameter versions of the algorithm and a combination with nearest-neighbour construction for the TSP[2,8,9], as well as new local searches for related routing problems[10]. Combining a population-based method with a local search such as 2-opt[11] is the idea behind memetic algorithms[12] and is standard practice for the TSP[13]. Iterated local search (ILS)[14,15] is the natural reference method for such hybrids, because it uses the same local search with a much simpler control structure. Specialised solvers based on the Lin–Kernighan heuristic[16] solve instances of the size considered here to optimality or near-optimality within seconds, so a new metaheuristic for the TSP is better justified as a study of how a search framework and a local search interact than by raw performance alone.

Evaluations of nature-inspired methods are often criticised when new methods are not compared with simple, equally equipped baselines and when the contribution of individual components is not isolated[17]. We therefore study a bees-algorithm hybrid, HBALS, with three design principles: the

comparison must give every method the same local search and the same computational budget; the contribution of each component must be measured by ablation; and conclusions must rest on appropriate non-parametric statistics. In developing HBALS we encountered several pitfalls that are common in such studies and that we document and quantify: baselines without local search inflate the advantage of a hybrid; an adaptive neighbourhood rule can be silently inactive (here because of integer truncation) or harmful once it works; a step labelled "3-opt-style" was in fact a random segment reversal; and an early-stopping rule ended runs far before the nominal iteration limit.

We report the following. Under equal budget, HBALS ranks first among six methods on 12 benchmark instances with 51–198 cities and is clearly better than four of them, but it is statistically indistinguishable from ILS, which is better on larger instances. Local search at the elite sites and a single-swap perturbation are the components that matter. The study therefore does not claim a new state-of-the-art solver; it provides a reproducible assessment of what a bees-style hybrid contributes.

Results

Comparison under equal budget

Every method received the same budget of move evaluations on each instance and, where applicable, the same 2-opt operator (Methods). The budget equals what 300 iterations of HBALS consume, from 39 million evaluations on eil51 to 754 million on d198 (Supplementary Table S2). Table 1 gives the mean gap to the best-known tour length over 20 runs on 12 instances.

Table 1 Mean gap to the best-known tour length (%) under equal budget; standard deviation over runs in parentheses; lowest mean per instance in bold. n = 20 runs per cell (n = 10 for BA). Average ranks are over the 12 instances (Friedman test, see text).

Instance	N	HBALS	ILS-2opt	GA-2opt	ACO-2opt	MS-2opt	BA
eil51	51	**0.035 (0.09)**	0.200 (0.17)	0.317 (0.34)	0.739 (0.26)	2.183 (0.55)	22.160 (3.57)
berlin52	52	**0.000 (0.00)**	0.154 (0.69)	0.154 (0.69)	0.036 (0.05)	1.784 (0.77)	25.700 (4.81)
st70	70	**0.030 (0.08)**	0.348 (0.62)	0.385 (0.42)	1.200 (0.33)	2.474 (0.61)	45.659 (6.72)
eil76	76	**0.158 (0.24)**	0.316 (0.63)	0.641 (0.56)	1.980 (0.56)	4.238 (0.50)	32.677 (3.61)
kroA100	100	**0.008 (0.03)**	0.117 (0.13)	0.148 (0.22)	2.314 (0.53)	4.346 (0.74)	66.527 (6.04)
kroB100	100	**0.237 (0.17)**	0.351 (0.20)	0.301 (0.27)	1.769 (0.40)	4.834 (0.70)	59.533 (6.08)
eil101	101	**0.175 (0.18)**	0.350 (0.58)	0.723 (0.50)	4.976 (0.59)	4.897 (0.65)	39.269 (3.10)
lin105	105	**0.160 (0.22)**	0.208 (0.25)	0.379 (0.45)	1.418 (0.36)	5.049 (0.64)	70.529 (10.54)
ch130	130	**0.601 (0.36)**	0.676 (0.46)	0.903 (0.61)	4.238 (0.61)	5.160 (0.66)	65.290 (7.30)
ch150	150	0.545 (0.17)	**0.541 (0.44)**	0.722 (0.54)	3.377 (0.95)	7.088 (0.94)	73.116 (5.96)
rat195	195	1.184 (0.23)	**0.837 (0.44)**	1.539 (0.54)	5.712 (0.81)	8.762 (0.93)	81.373 (6.02)
d198	198	0.461 (0.18)	**0.324 (0.16)**	0.554 (0.22)	6.163 (1.03)	4.523 (0.38)	100.734 (15.18)
Average rank		**1.25**	**1.96**	**2.96**	**4.00**	**4.83**	**6.00**

N, number of cities; ILS, iterated local search; GA, genetic algorithm; ACO, ant colony optimisation; MS, multi-start; BA, plain bees algorithm without local search.

HBALS had the lowest mean gap on 9 of the 12 instances and the best average rank (1.25), followed by ILS-2opt (1.96), GA-2opt (2.96), ACO-2opt (4.00), multi-start 2-opt (4.83) and BA (6.00). The ranks differ overall (Friedman test, n = 12 instances, six methods, $\chi^2(5) = 55.0$, $p = 1.3\times10^{-10}$). The Nemenyi critical difference at $\alpha = 0.05$ is 2.18, so in terms of rank HBALS is distinguishable from ACO-2opt, multi-start 2-opt and BA, but not from ILS-2opt or GA-2opt. Paired comparisons over the 12 instances are more sensitive: HBALS had a lower mean gap than GA-2opt, ACO-2opt, multi-start 2-opt and BA on

all 12 instances (two-sided Wilcoxon signed-rank tests, n = 12 pairs, Holm-adjusted p = 0.0024 for each; Supplementary Table S4), whereas against ILS-2opt it was better on 9 and worse on 3 instances, which is not a significant difference (p = 0.129).

Run-level tests give the same picture (two-sided Mann–Whitney U tests, n = 20 runs per method, Holm-adjusted over the 12 instances; Supplementary Table S3). HBALS was better than ILS-2opt on eil51 (adjusted p = 0.011) and kroA100 (adjusted p = 0.004). ILS-2opt had the lower mean on rat195 (adjusted p = 0.055) and d198 (adjusted p = 0.134), but these differences did not reach α = 0.05. Once all population-based methods use the same 2-opt operator, the differences between them are small compared with the difference between methods with and without local search: GA-2opt and ACO-2opt reach mean gaps between 0.04% and 6.2%, whereas the plain BA without local search stays between 22% and 101%.

Behaviour within the budget

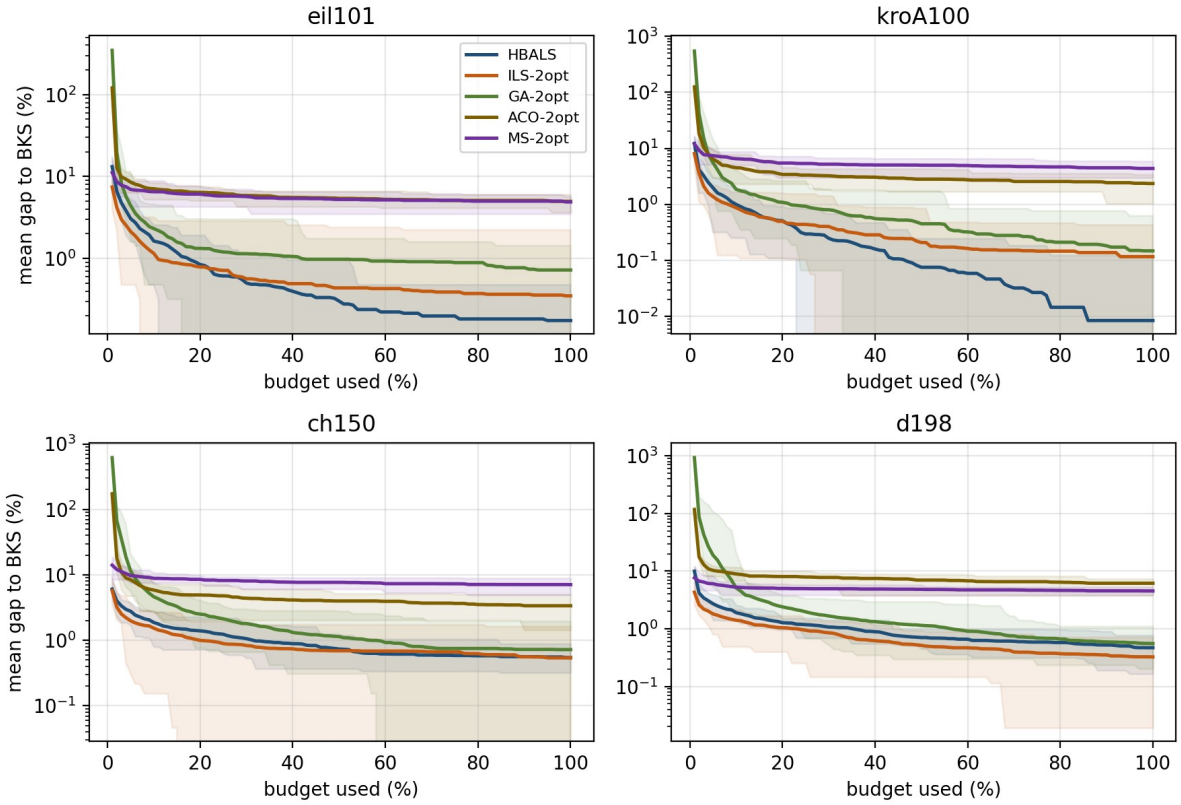


Figure 1 Anytime behaviour. Mean best-so-far gap to the best-known tour length (logarithmic axis) as a function of the fraction of the evaluation budget used; shaded bands show the minimum–maximum range over 20 runs. ILS-2opt is ahead during the first part of the budget on most instances, whereas HBALS overtakes it on eil101 and kroA100 and stays close on ch150 and d198.

The anytime curves (Fig. 1) show that the ranking between HBALS and ILS depends on how much of the budget is used: ILS-2opt improves faster at the start, and HBALS catches up and, on the smaller instances, overtakes it later.

Larger instances

Table 2 Mean gap to the best-known tour length (%) on three larger instances; n = 10 runs per cell. The last column is the Holm-adjusted p value of a two-sided Mann–Whitney U test of HBALS against ILS-2opt (three instances).

Instance	N	HBALS	ILS-2opt	GA-2opt	ACO-2opt	Adj. p
kroA200	200	**0.46**	0.71	1.42	11.29	0.384
a280	280	2.58	**0.66**	1.79	20.40	0.002
lin318	318	2.60	**1.72**	2.61	13.21	0.003

On kroA200 the mean gap of HBALS was lower than that of ILS-2opt, but not significantly (adjusted p = 0.38). On a280 and lin318, ILS-2opt was better (adjusted p = 0.0022 and 0.0034), and on a280 HBALS was also behind GA-2opt. Under this budget HBALS therefore does not scale as well as ILS. A plausible explanation, which we did not test, is that the cap of 20 improving 2-opt moves per candidate prevents the nearest-neighbour and perturbed tours from reaching a local optimum on larger instances, whereas ILS applies 2-opt until convergence.

Ablation study

Table 3 Ablation. Mean gap to the best-known tour length (%), n = 20 runs per cell. The last column gives the number of instances (of 4) on which the variant is significantly worse / significantly better than the final HBALS (two-sided Mann–Whitney U tests, Holm-adjusted over the seven variants, $\alpha = 0.05$).

Variant	st70	kroA100	ch130	d198	Average	Worse / better
HBALS (final)	0.030	0.008	0.601	0.461	0.275	–
+ adaptive rule	0.644	0.604	1.801	1.070	1.030	4 / 0
3 swaps per candidate	0.326	0.999	2.971	2.450	1.687	4 / 0
− nearest-neighbour seeding	0.096	0.020	0.676	0.657	0.362	1 / 0
− segment reversal	0.133	0.012	0.440	0.449	0.259	0 / 0
− selected-site 2-opt	0.081	0.000	0.496	0.422	0.250	0 / 0
− elite-site 2-opt	0.178	0.125	1.544	1.426	0.818	4 / 0
− scouts	0.096	0.008	0.615	0.438	0.290	0 / 0

Seven variants of HBALS were compared with the final configuration on four instances (Table 3 and Fig. 2). Two changes were clearly harmful: an adaptive rule that increases the number of random swaps after stagnation, and a fixed perturbation with three swaps. Both were significantly worse than the final HBALS on all four instances (adjusted $p \le 1.4\times10^{-6}$ and $\le 8.0\times10^{-5}$), with average gaps of 1.03% and 1.69% against 0.27%. With 2-opt as the repair operator, a single swap is an effective small perturbation, and larger perturbations destroy too much structure. Removing 2-opt at the elite sites was also significantly worse on all four instances (adjusted $p \le 0.0016$; average gap 0.82%). Removing nearest-neighbour seeding gave a small loss that was significant only on d198 (adjusted p = 0.019). The random segment reversal, the 2-opt at selected sites and the scout bees had no measurable effect: all adjusted p values were at least 0.10, and removing the reversal or the selected-site 2-opt even lowered the average gap slightly. With four instances the ablation has limited power, so small effects may have been missed.

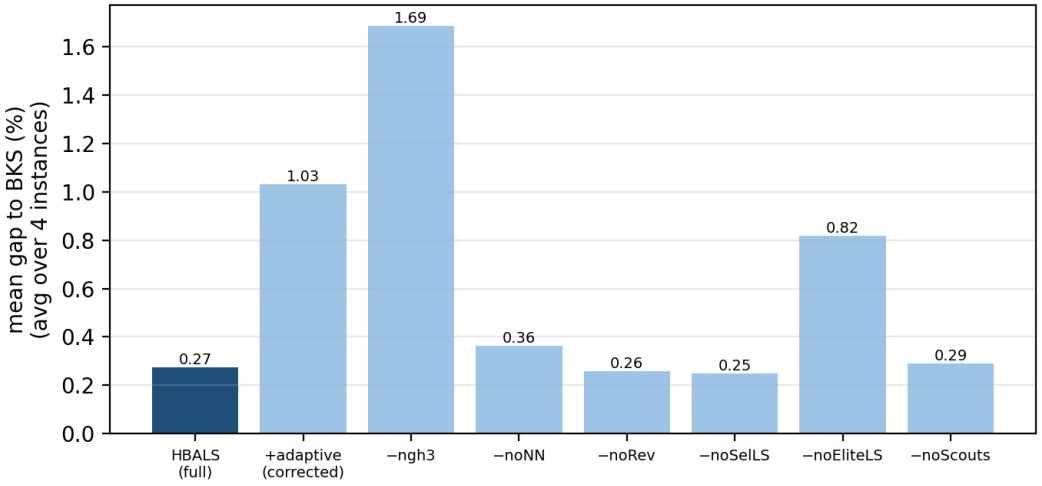


Figure 2 Average of the mean gap to the best-known tour length over the four ablation instances (st70, kroA100, ch130, d198) for each HBALS variant; n = 20 runs per instance.

Effect of early stopping

A common stopping rule ends a run after 20 consecutive iterations without improvement of the best tour (with at most 300 iterations). For HBALS this rule ended runs after only 38 to 94 iterations on average. Compared with the full budget, the average mean gap over the 12 instances rose from 0.30% to 0.95% (Supplementary Table S5 and Supplementary Fig. S2). Early stopping therefore substantially understates the quality that the algorithm reaches within its budget.

Comparison with published bees-algorithm results

Published results for bees-algorithm variants on overlapping instances[8,9] were not re-run, and their budgets, hardware and stopping rules differ, so the comparison is indicative only (Supplementary Table S6). The mean gap of HBALS was lower than the reported values of Sahin[9] on berlin52, st70 and kroA100, and higher on ch150 and rat195; it was lower than the reported values of Suluova and Pham[8] on every instance where both exist.

Discussion

Under equal budget and equal local search, HBALS is a strong member of the 2-opt-based family. It was better than memetic genetic and ant algorithms, multi-start 2-opt and a plain bees algorithm on all 12 instances (Holm-adjusted $p = 0.0024$), and on instances up to about 130 cities it was better than or indistinguishable from ILS, with significant advantages on eil51 and kroA100. The ablation attributes this performance mainly to two elements, 2-opt at the elite sites and a minimal single-swap perturbation, which is consistent with the view that the quality of the local search dominates the effect of the surrounding metaheuristic[13].

The evidence does not show that the bees-algorithm structure is necessary. ILS, a much simpler method, was better on the larger instances and ahead during the early part of the budget. Selected-site 2-opt, scout bees and the segment-reversal step could be removed without detectable loss, and the adaptive neighbourhood was harmful. The difference between HBALS and ILS over the 12 main instances was not significant. The contribution is therefore best described as an empirically validated hybrid configuration with a component analysis, not as a new state-of-the-art TSP solver; specialised solvers[16] are far more effective on these instances. Two opposite biases affect evaluations of this kind: baselines without local

search inflate the advantage of a hybrid, whereas early stopping inflates its mean gap (here by roughly a factor of three).

The study has limitations. The instances are Euclidean TSPLIB instances with at most 318 cities, so behaviour on clustered, asymmetric or much larger instances is unknown. Budgets are tied to the nominal budget of 300 HBALS iterations, and the anytime curves suggest that the ranking against ILS depends on the budget. The 2-opt operator restarts its scan after each improving move and uses no neighbour lists, and its move limits (20 and 10) were not tuned; a more efficient local search would probably change all methods, particularly on large instances. HBALS and the baselines were not tuned, and a preliminary exploratory check of neighbourhood rules was made on one instance (kroA100). The ablation used four instances and changed one component at a time, so interactions were not studied, and the larger instances used 10 runs only. Future work could use a size-dependent move limit or a stronger local search, a localised perturbation instead of more random swaps, a simplified HBALS without the components that had no effect, and other budgets and instance classes.

Methods

Problem, instances and performance measure

A tour is a permutation π of N cities, and its length is

$$L(\pi) = d(\pi_N, \pi_1) + \Sigma_i\, d(\pi_i, \pi_{i+1}), \quad i = 1, \ldots, N - 1 \qquad (1)$$

where d(i, j) is the distance between cities i and j. Distances follow the TSPLIB convention (Euclidean distance rounded to the nearest integer)[18]. Twelve instances with 51 to 198 cities form the main set (eil51, berlin52, st70, eil76, kroA100, kroB100, eil101, lin105, ch130, ch150, rat195, d198), and three larger instances (kroA200, a280, lin318) test scaling. Best-known tour lengths (BKS) are the published optima[18]; for ten of the 15 instances (eil51, berlin52, st70, eil76, kroA100, eil101, lin105, ch130, ch150, a280) an optimal-tour file was available and the BKS was verified by recomputing its length, and for the others (kroB100, rat195, d198, kroA200, lin318) the TSPLIB values were used. No run in this study produced a tour shorter than the BKS. The performance measure of a run is the gap 100 × (L − BKS) / BKS, where L is the length of the best tour of the run. Apart from a short exploratory check of neighbourhood rules on kroA100 (10 runs, different random seeds), no tuning was done on these instances.

HBALS

HBALS maintains a population of $n_b = 20$ tours, ranks it by tour length at every iteration and treats the best e = 3 tours as elite sites and the best m = 12 as selected sites. Each site is explored by recruited bees; each bee perturbs a copy of the site and may improve it by local search, and the best tour found at a site replaces the site. The remaining $n_b - m$ members of the next population are random scout tours.

Initialisation. The first ten tours are nearest-neighbour tours started from cities 0 to 9; the other ten are random permutations. **Elite sites.** Each elite site receives nep = 8 candidates. A candidate is the site with k = 1 random swap applied; with probability 0.7 a first-improvement 2-opt search (at most 20 improving moves) is applied, and with probability 0.1 a random segment-reversal step is applied (three random segment reversals are evaluated independently and the best improving one is kept). **Selected sites.** Each remaining selected site receives nsp = 4 candidates with one random swap and, with probability 0.3, a 2-opt search of at most 10 improving moves. **Scouts and termination.** Random tours complete the population, the global best tour is updated after every iteration, and the search ends when the evaluation budget is exhausted.

The 2-opt operator scans pairs of edges in index order, applies the first move that shortens the tour and restarts the scan, until no improving move exists or the move limit is reached. This restart-based implementation is simple but not efficient (no neighbour lists or don't-look bits); it is used identically by every method with local search. The "segment-reversal" step is easily mistaken for a lightweight 3-opt move: it draws three indices but reverses a single segment between two of them, so it is a randomised 2-opt-type move. The neighbourhood size is fixed at k = 1 because an adaptive rule was harmful (Results). In a preliminary implementation the adaptive rule, which shrinks k by a factor 0.9 after improvements and expands it by 1.05 after three non-improving iterations, was applied with integer truncation, so k fell from 3 to 1 within two iterations and could not grow again; the mechanism was inactive, and the method is therefore not called adaptive.

Algorithm 1 Pseudocode of HBALS.

```
Input: D; n_b, m, e, nep, nsp, k; p_el, p_sel, p_rev; budget B
 1  initialise n_b tours (first half nearest-neighbour, rest random)
 2  best <- best tour of the population
 3  while evaluations < B:
 4      rank the population by tour length
 5      for each elite site s (best e):
 6          repeat nep times: c <- s with k random swaps
 7              with prob. p_el: 2-opt on c (<= 20 moves)
 8              with prob. p_rev: random segment reversal on c
 9          s <- best of s and its candidates
10      for each other selected site s (ranks e+1..m):
11          repeat nsp times: c <- s with max(1, floor(k/2)) swaps
12              with prob. p_sel: 2-opt on c (<= 10 moves)
13          s <- best of s and its candidates
14      add n_b - m random scout tours
15      update best with the best tour of the new population
16  return best
```

Table 4 Parameters of the final HBALS configuration. The values are those of the "fast preset" of the accompanying implementation and were not tuned on the benchmark instances used for reporting.

Parameter	Value	Description
n_b	20	Population size
m / e	12 / 3	Selected sites / elite sites
nep / nsp	8 / 4	Candidates per elite site / per non-elite selected site
k	1	Random swaps per elite candidate (max(1, ⌊k/2⌋) at selected sites)
nn_ratio	0.5	Fraction of nearest-neighbour tours in the initial population
p_{el} / p_{sel} / p_{rev}	0.7 / 0.3 / 0.1	Probability of 2-opt at elite sites / at selected sites / of the segment reversal
2-opt move limit	20 / 10	Maximum improving moves per candidate at elite / selected sites

Equal-budget protocol

Giving all methods the same number of iterations is unfair, because an iteration of HBALS (with 2-opt on many candidates) is far more expensive than an iteration of a plain genetic algorithm. Each run is therefore limited by a budget of move evaluations. One unit is one incremental move evaluation (a 2-opt or segment-reversal delta counts as 1 and a swap delta as 4); a full tour-length computation counts as N,

one ant tour construction as $N^2/2$ and one order crossover as N. This count is machine independent. The budget of an instance is the mean number of evaluations consumed by 300 iterations of HBALS without early stopping (three pilot runs of the preliminary configuration, rounded to 10^5); the budgets are listed in Supplementary Table S2. All methods stop when the budget is exhausted (the last iteration may overshoot it by less than 1%), and HBALS has no early stopping.

Compared methods

ILS-2opt starts from a nearest-neighbour tour, applies a double-bridge kick and 2-opt to a local optimum, and accepts the result if it is not worse. GA-2opt is a memetic genetic algorithm (population 20, tournament selection of size 3, order crossover with probability 0.8, swap mutation with probability 0.1, elitism of 2) with 2-opt of at most 20 moves applied to every offspring. ACO-2opt is an ant system (20 ants, $\alpha = 1$, $\beta = 2$, evaporation 0.5, $Q = 100$, initial pheromone 0.1) with 2-opt of at most 20 moves applied to every ant tour. Multi-start 2-opt repeatedly applies 2-opt to random tours and keeps the best result. The plain bees algorithm (BA) uses random initialisation, the same m, e, nep and nsp as HBALS, three single-swap neighbours per bee and scouts, without local search. Details are given in Supplementary Table S1. A particle swarm baseline is not included, to keep the number of memetic baselines manageable.

Statistical analysis

Tour lengths are discrete and not normally distributed, so non-parametric tests were used ($\alpha = 0.05$, two-sided). Per instance, methods were compared by Mann–Whitney U tests on the final tour lengths of n = 20 runs (n = 10 for BA and for the larger instances), implemented in SciPy with its default method selection, together with the Vargha–Delaney A12 effect size[19]. Across instances, the Friedman test (n = 12 instances, six methods) with a Nemenyi critical difference[20,21] was used for ranks, and HBALS was compared with each other method by Wilcoxon signed-rank tests on the 12 per-instance mean gaps. P values were adjusted by the Holm procedure[22] within each family of comparisons: HBALS against the five other methods (12 instances), HBALS against ILS-2opt per instance (12 instances), the three larger instances, and the seven ablation variants within each instance. Descriptive statistics are the mean and the standard deviation over runs. Exact p values are reported.

Implementation and reproducibility

All methods were implemented in Python with Numba, and each run used one CPU core and the seed 1000 + run index. The complete study (2,240 analysed runs) was executed on a Windows 11 personal computer (Python 3.12.7, NumPy 1.26.4, SciPy 1.13.1, Numba 0.60.0) and, independently, in a Linux container (Python 3.12.3, NumPy 2.4.4, SciPy 1.17.1, Numba 0.68.0). The final tour length, the number of evaluations and the number of iterations were identical in every run, so the results do not depend on the platform or on these library versions. The Numba implementation of the preliminary pure-Python configuration (integer-truncated neighbourhood, early stopping after 20 stagnant iterations, at most 300 iterations) reproduced the six means obtained with the original pure-Python implementation (eil51 427.4 versus 427.9, berlin52 7543.2 versus 7543.0, st70 677.9 versus 679.4, kroA100 21408 versus 21389, ch150 6614 versus 6628, rat195 2381 versus 2375; 20 runs each), which supports the fidelity of the port. Scripts, instances, all per-run results and anytime curves are available in the repository.

Use of AI tools

ChatGPT (OpenAI) and Claude (Anthropic) were used for language editing, for feedback on the manuscript and repository documentation, and for assistance in implementing the algorithms in Numba and in writing the analysis scripts. The study was first executed by the AI assistant in a sandbox and then re-executed in full by the author on his own computer, with identical results. The author reviewed and

verified the code, the results and the text and takes full responsibility for the work. No AI tool is an author.

Data availability

The benchmark instances are available from TSPLIB[18]. All per-run results, anytime curves, summary tables and the code that produced them are available at https://github.com/ErfanKhadiv/HBALS-TSP (directory v2) and are archived, together with the preprint, at Zenodo (https://doi.org/10.5281/zenodo.23450861). Results for published methods in Supplementary Table S6 are taken from the original publications[8,9] and were not re-run.

Code availability

The source code (Python, Numba), run scripts and analysis scripts are available at https://github.com/ErfanKhadiv/HBALS-TSP and archived at Zenodo (https://doi.org/10.5281/zenodo.23450861).

References

1. Blum, C. & Roli, A. Metaheuristics in combinatorial optimization: overview and conceptual comparison. *ACM Comput. Surv.* **35**, 268–308 (2003).
2. Hartono, N., Suluova, H. F., Eker, F. M., Zeybek, S. & Caterino, M. A comparative study of reduced parameter versions of the bees algorithm for traveling salesman problem. *J. Integr. Syst.* **7**, 1–12 (2024).
3. Pop, P. C., Cosma, O., Sabo, C. & Sitar, C. P. A comprehensive survey on the generalized traveling salesman problem. *Eur. J. Oper. Res.* **314**, 819–835 (2024).
4. Pham, D. T. et al. The bees algorithm—a novel tool for complex optimisation problems. In *Proc. 2nd Virtual Int. Conf. Intelligent Production Machines and Systems (IPROMS)* 454–459 (2006).
5. Ismail, A. H., Ruslan, W. & Pham, D. T. A user-friendly bees algorithm for continuous and combinatorial optimisation. *Cogent Eng.* **10**, 2278257 (2023).
6. Pham, D. T., Castellani, M., Sholedolu, M. & Ghanbarzadeh, A. The bees algorithm and mechanical design optimisation. In *Proc. 5th Int. Conf. Informatics in Control, Automation and Robotics (ICINCO-ICSO)* 250–255 (2008).
7. Yuce, B., Packianather, M. S., Mastrocinque, E., Pham, D. T. & Lambiase, A. Honey bees inspired optimization method: the bees algorithm. *Insects* **4**, 646–662 (2013).
8. Suluova, H. F. & Pham, D. T. A new single-parameter bees algorithm. *Biomimetics* **9**, 634 (2024).
9. Sahin, M. Solving TSP by using combinatorial bees algorithm with nearest neighbor method. *Neural Comput. Appl.* **35**, 1863–1879 (2023).
10. Hamza, A., Haj Darwish, A. & Rihawi, O. A new local search for the bees algorithm to optimize multiple traveling salesman problem. *Intell. Syst. Appl.* **18**, 200242 (2023).
11. Croes, G. A. A method for solving traveling-salesman problems. *Oper. Res.* **6**, 791–812 (1958).
12. Moscato, P. On evolution, search, optimization, genetic algorithms and martial arts: towards memetic algorithms. Caltech Concurrent Computation Program Report 826 (1989).
13. Johnson, D. S. & McGeoch, L. A. The traveling salesman problem: a case study in local optimization. In *Local Search in Combinatorial Optimization* (eds Aarts, E. H. L. & Lenstra, J. K.) 215–310 (Wiley, 1997).
14. Lourenço, H. R., Martin, O. C. & Stützle, T. Iterated local search. In *Handbook of Metaheuristics* (eds Glover, F. & Kochenberger, G. A.) 320–353 (Springer, 2003).

15. Martin, O., Otto, S. W. & Felten, E. W. Large-step Markov chains for the traveling salesman problem. *Complex Syst.* **5**, 299–326 (1991).

16. Helsgaun, K. An effective implementation of the Lin–Kernighan traveling salesman heuristic. *Eur. J. Oper. Res.* **126**, 106–130 (2000).

17. Sörensen, K. Metaheuristics—the metaphor exposed. *Int. Trans. Oper. Res.* **22**, 3–18 (2015).

18. Reinelt, G. TSPLIB—a traveling salesman problem library. *ORSA J. Comput.* **3**, 376–384 (1991).

19. Vargha, A. & Delaney, H. D. A critique and improvement of the CL common language effect size statistics of McGraw and Wong. *J. Educ. Behav. Stat.* **25**, 101–132 (2000).

20. Demšar, J. Statistical comparisons of classifiers over multiple data sets. *J. Mach. Learn. Res.* **7**, 1–30 (2006).

21. Derrac, J., García, S., Molina, D. & Herrera, F. A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm Evol. Comput.* **1**, 3–18 (2011).

22. Holm, S. A simple sequentially rejective multiple test procedure. *Scand. J. Stat.* **6**, 65–70 (1979).

Acknowledgements

The author thanks Dr Masoud Bekravi for supervising the Computer Project course at Islamic Azad University, Ardabil, under which this work was carried out.

Author contributions

E.K. conceived and designed the study, implemented the software, performed the experiments, analysed the results and wrote the manuscript.

Competing interests

The author declares no competing interests.

Additional information

Funding. No funding was received for this study. **Ethics.** This study did not involve human participants or animals. **Correspondence** and requests for materials should be addressed to E.K. (erfan.khadiv.dev@gmail.com). **Supplementary information** accompanies this manuscript.